

Test Unitarios en Python



Testing Unitario con Pytest



- ¿Qué es el Unit Testing?
- Introducción a PyTest y sus ventajas
- Estructura del proyecto para pruebas
- Fixtures y Mocks en PyTest
- Ejemplo práctico de pruebas unitarias



¿Qué es Unit Testing?

- Proceso de verificación individual de componentes de software.
- Asegura el correcto funcionamiento de cada unidad.
- Detecta errores en etapas tempranas del desarrollo.
- Facilita la refactorización y mejora del código.
- Garantiza la calidad y estabilidad del software.

Introducción a Pytest

- Pytest: Framework para pruebas unitarias.
- Simplifica la escritura y ejecución de tests.
- Detección automática de pruebas.
- Salidas de error claras y concisas.
- Instalación vía pip: `pip install pytest`.



Estructura de un Proyecto

```
setup.py
src/
    mypkg/
        __init__.py
        app.py
        view.py
tests/
    __init__.py
    unit/
        __init__.py
        app_test.py
    integration/
        __init__.py
        app_test.py
```

- Organización de directorios para pruebas.
- Convención de nombres para archivos de prueba.
- Funciones de prueba: Nombres significativos.
- Estructura de un proyecto con PyTest.

Primera prueba con Pytest

- Crear una función simple como ejemplo.
- Escribir la prueba usando Pytest.
- Usar 'assert' para verificar resultados.
- Ejecutar pruebas desde la línea de comandos.



Primera prueba con Pytest

```
def sumar(a, b):  
    return a + b
```

```
from src.mi_modulo import sumar  
  
def test_sumar_dos_numeros():  
    assert sumar(2, 3) == 5
```



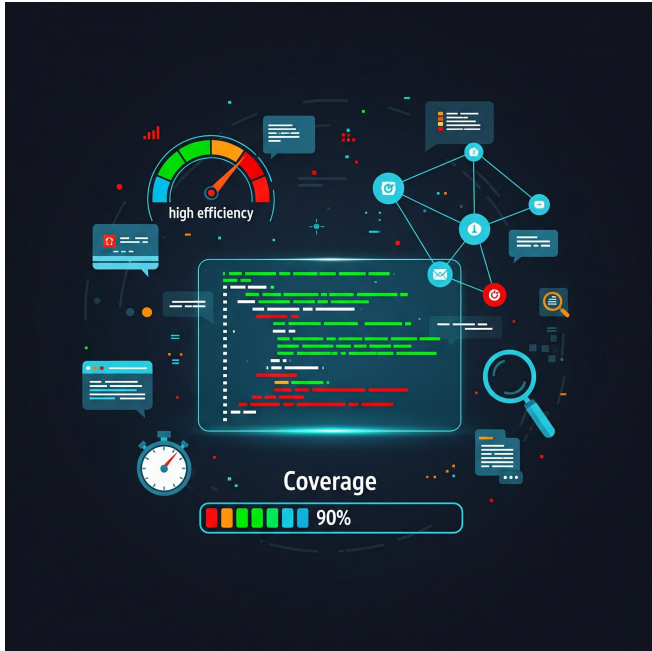
Tipos de Asserts en Pytest

- Pytest: Afirmaciones para verificar el comportamiento del código
- Tipos de 'asserts': Igualdad, pertenencia y más
- El mensaje de error de Pytest es muy útil
- Las pruebas fallidas proporcionan información detallada

Tipos de Asserts en Pytest

- **assert a == b:** Verifica si dos valores son iguales.
- **assert a != b:** Verifica si dos valores son diferentes.
- **assert a > b:** Verifica si a es mayor que b.
- **assert a < b:** Verifica si a es menor que b.
- **assert a >= b:** Verifica si a es mayor o igual que b.
- **assert a <= b:** Verifica si a es menor o igual que b.
- **assert item in collection:** Verifica si un item se encuentra en una collection (lista, tupla, cadena, etc.).
- **assert item not in collection:** Verifica si un item NO se encuentra en una collection.
- **assert boolean_expression:** Verifica si una expresión booleana es True.

Parámetros en Pytest



- Parametrización: Ejecuta una prueba con diferentes entradas.
- `@pytest.mark.parametrize`: Decorador para definir parámetros.
- Simplifica la escritura de múltiples casos de prueba.
- Mejora la cobertura del código y la eficiencia.

Parámetros en Pytest

```
1 import pytest
2 from figuras import Triangulo
3
4 @pytest.mark.parametrize(
5     "lado1, lado2, lado3, expected_area, expected_perimeter",
6     [
7         (3, 4, 5, 6.0, 12),
8         (5, 5, 8, 12.0, 18),
9         (7, 10, 5, 16.25, 21),
10    ]
11 )
12 def test_triangulo_parametrized(lado1, lado2, lado3, expected_area, expected_perimeter):
13     t = Triangulo(lado1, lado2, lado3)
14     assert round(t.superficie(), 2) == round(expected_area, 2)
15     assert t.perimetro() == expected_perimeter
16
```

Reportes PyTest

- Ejecuta pruebas con Pytest desde la línea de comandos.
- Usa `pytest -k "nombre"` para ejecutar pruebas específicas.
- Usa `pytest -v` para una salida más detallada de los resultados.
- Genera reportes de pruebas para análisis.

```
. █ ..... █  
  
=== 10 passed, 2 skipped, 1 warnings in 2.53s ===  
  
--- detailed test report ---  
  
test_user_login_success  
test_user_invalid_password  
test_user_invalid_password  
350  
PASSED 237507  
test_product_inventory_update_update  
20073  
FAILED 24377  
PASSED 25630  
FAILED traceback : Expected_400  
  
//AssertionError: Expected 200 200 but got 401  
  
$
```

Conclusiones

- Pytest facilita el Unit Testing con simplicidad y flexibilidad.
- Las pruebas unitarias mejoran la calidad del software.
- Pytest agiliza la detección temprana de errores en el código.
- Es valiosa la integración de Pytest con herramientas de CI/CD (Github, Gitlab, etc).
- Automatiza la ejecución de pruebas para asegurar la calidad.





¡Gracias!

Hasta la próxima

